

《 Introduction to Embedded System 》

Lab-3

Major: Software Engineering

Name: Md Tasnimul Hasan

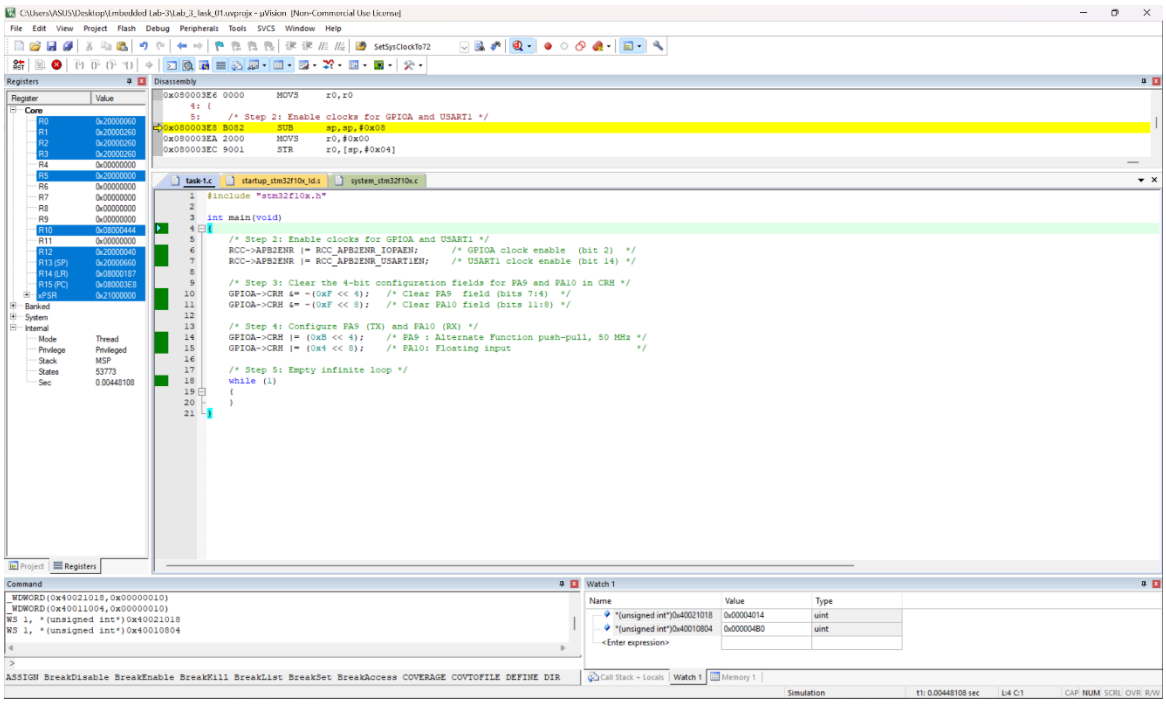
Student ID: 202480090104

Task 1: Enable Clocks and Configure the USART1 Pins

Objective: To enable peripheral clocks and configure GPIO pins (PA9 and PA10) for USART1 communication.

Description: Successfully enabled the GPIOA and USART1 clocks by setting the respective bits in RCC->APB2ENR. Configured PA9 as an Alternate Function push-pull (50 MHz) for TX and PA10 as a floating input for RX. Verified the bit fields in the Watch window, confirming the CRH register correctly reflected the intended pin configurations.

Code and Output:

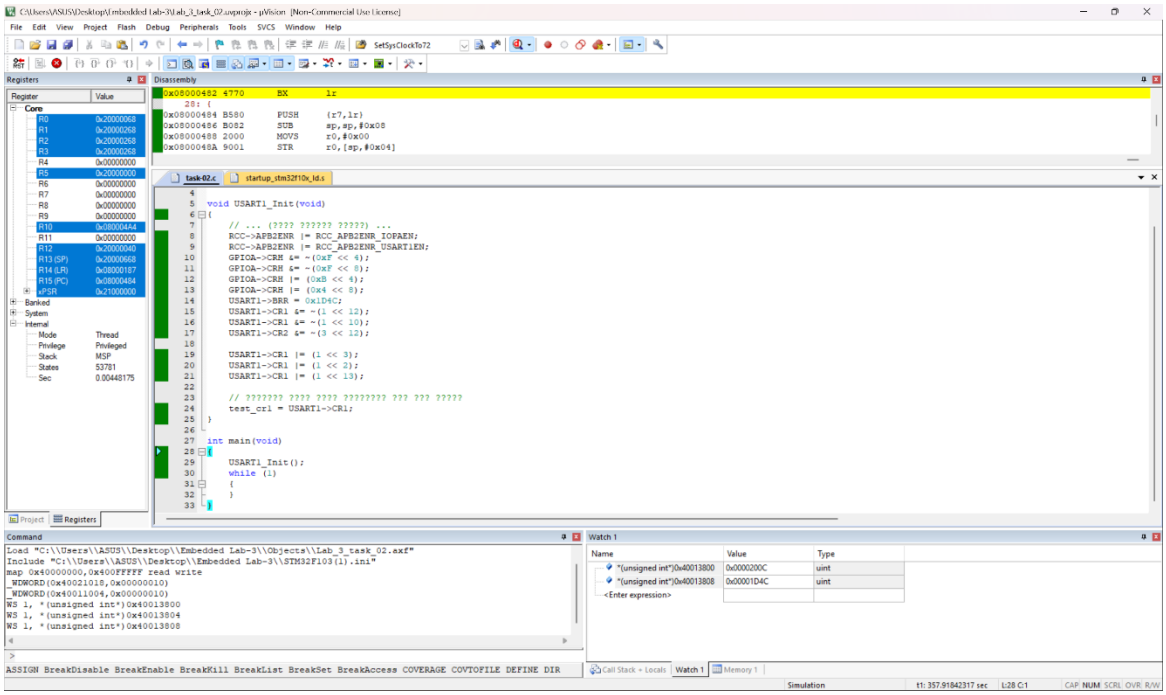


Task 2: Configure the 8N1 Frame and Enable USART1

Objective: To initialize USART1 with a specific baud rate and 8N1 frame format.

Description: Created the USART1_Init() function to set the baud rate to 9600 using BRR = 0x1D4C. Cleared the M, PCE, and STOP bits to achieve 8N1 configuration. Finally, enabled the UE, TE, and RE bits in the CR1 register. Verified via Watch window that CR1 displayed 0x0000200C.

Code and Output:

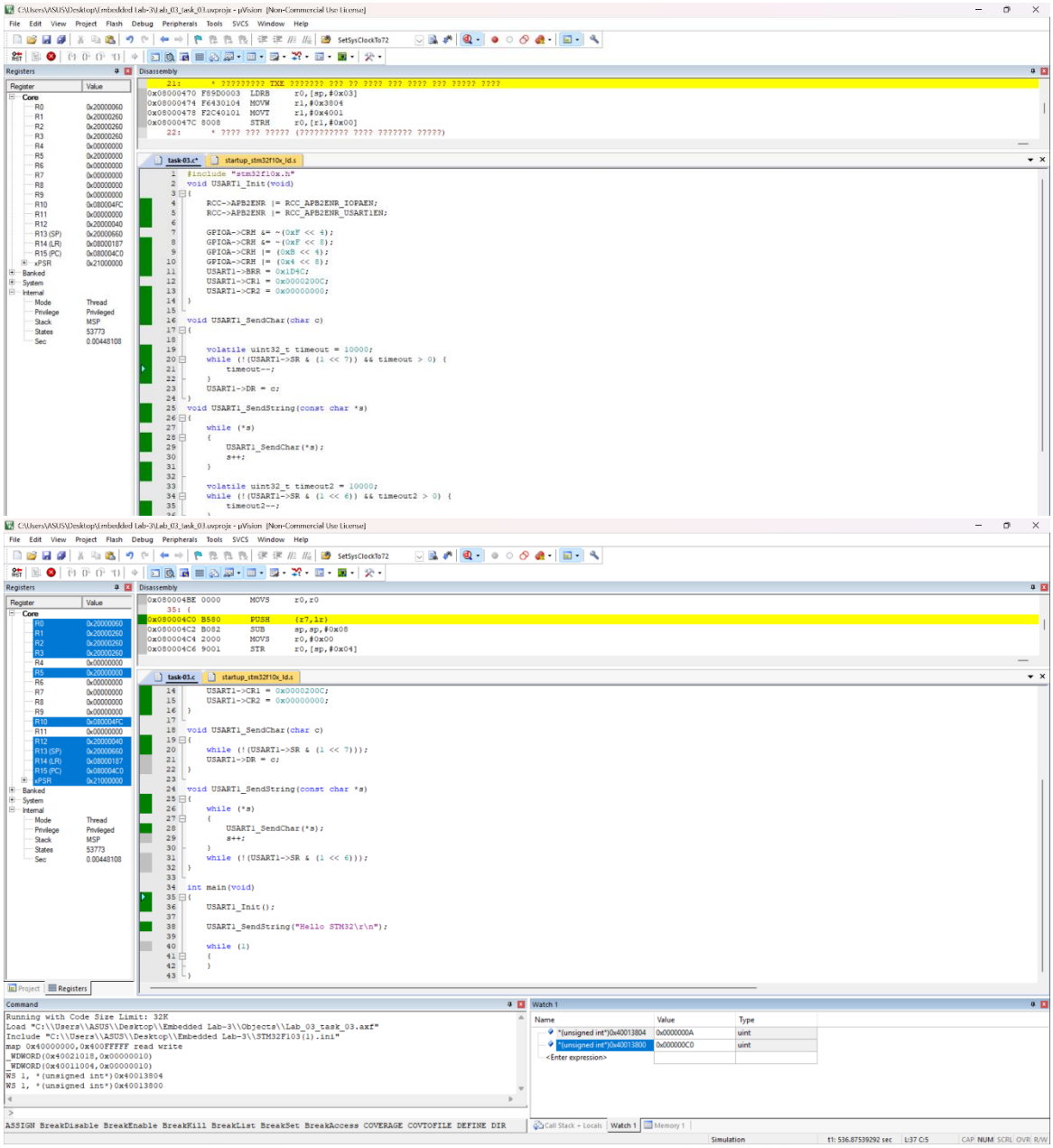


Task 3: Transmit a String

Objective: To transmit string data over USART1 using polling method.

Description: Implemented USART1_SendChar() and USART1_SendString() functions. The TXE flag was polled before sending each character, and the TC flag was checked at the end. Transmitted "Hello STM32\r\n". The ASCII hex values were confirmed in the DR register (e.g., 0x48 for 'H').

Code and Output:

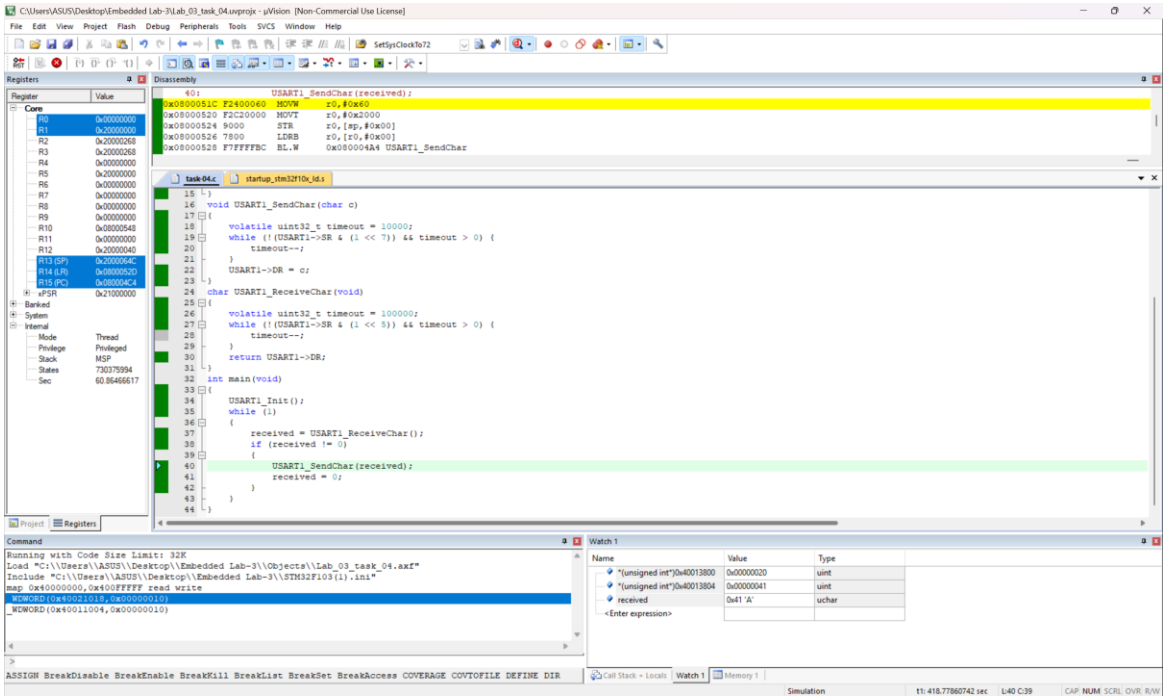


Task 4: Receive a Character and Echo It Back

Objective: To implement character reception and echo functionality using polling.

Description: Developed USART1_ReceiveChar() to poll the RXNE flag. In the main loop, received characters were stored in a global variable and immediately echoed back. Due to simulator limitations, incoming data was simulated by manually writing to DR and SR. Verified successful echo by observing the DR register.

Code and Output:

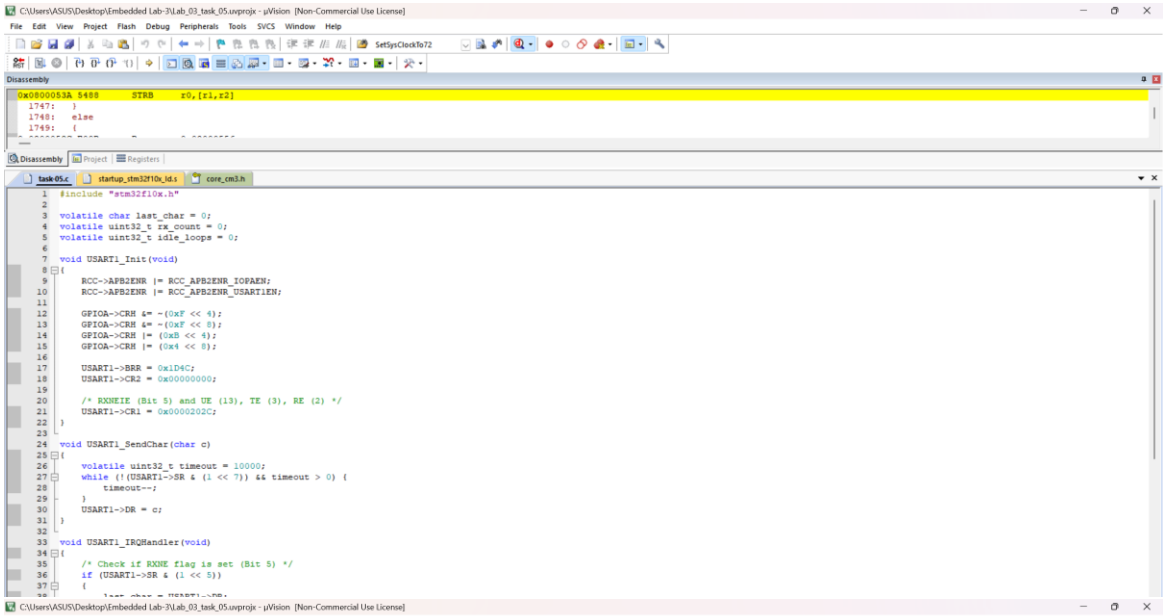


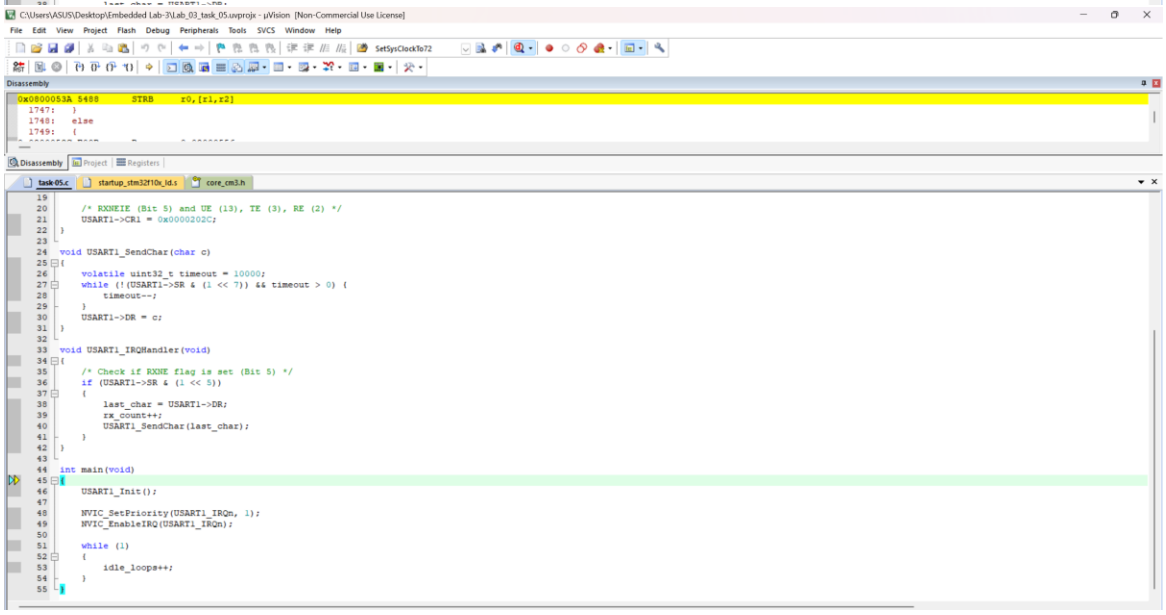
Task 5: Interrupt-Driven Reception with RXNE

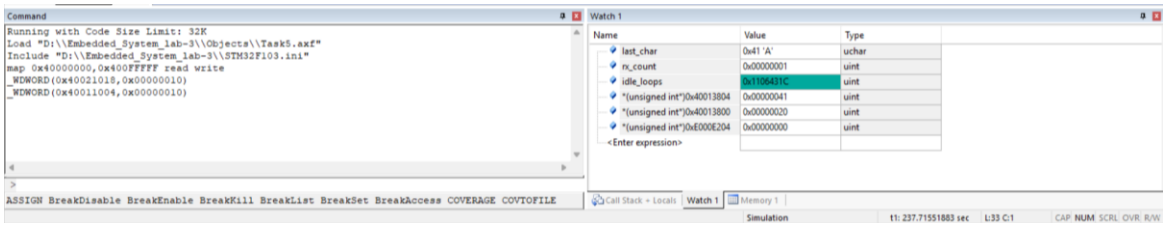
Objective: To implement interrupt-driven serial communication.

Description: Enabled the RXNEIE interrupt in USART1 and configured the NVIC. The USART1_IRQHandler reads the DR, stores the character, increments the counter, and echoes the data. Observed that idle_loops continuously increased in the main loop, proving non-blocking operation, while interrupts were triggered manually.

Code and Output:


C:\Users\ASUS\Desktop\Embedded Lab-3\Lab_03_task_05.uvprojx - µVision [Non-Commercial Use License]
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
SetSysClock To72
Disassembly
0x0800053A 5488 STRB r0,[r1,r2]
1747: }
1748: else
1749: {
Disassembly Project Registers
task05.c startup_stm32f10x_ld.s core_cm3.h
#include "stm32f10x.h"
volatile char last_char = 0;
volatile uint32_t rx_count = 0;
volatile uint32_t idle_loops = 0;
void USART1_Init(void)
{
RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;
RCC->APB2ENR |= RCC_APB2ENR_USART1EN;
GPIOA->CRH &= ~(0xF << 4);
GPIOA->CRH &= ~(0xF << 8);
GPIOA->CRH |= (0xB << 4);
GPIOA->CRH |= (0x4 << 8);
USART1->BRR = 0x1D4C;
USART1->CR2 = 0x00000000;
/* RXNEIE (Bit 5) and UE (13), TE (3), RE (2) */
USART1->CR1 = 0x0000202C;
}
void USART1_SendChar(char c)
{
volatile uint32_t timeout = 10000;
while (!(USART1->SR & (1 << 7)) && timeout > 0) {
timeout--;
}
USART1->DR = c;
}
void USART1_IRQHandler(void)
{
/* Check if RXNE flag is set (Bit 5) */
if (USART1->SR & (1 << 5))
{


C:\Users\ASUS\Desktop\Embedded Lab-3\Lab_03_task_05.uvprojx - µVision [Non-Commercial Use License]
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
SetSysClock To72
Disassembly
0x0800053A 5488 STRB r0,[r1,r2]
1747: }
1748: else
1749: {
Disassembly Project Registers
task05.c startup_stm32f10x_ld.s core_cm3.h
/* RXNEIE (Bit 5) and UE (13), TE (3), RE (2) */
USART1->CR1 = 0x0000202C;
}
void USART1_SendChar(char c)
{
volatile uint32_t timeout = 10000;
while (!(USART1->SR & (1 << 7)) && timeout > 0) {
timeout--;
}
USART1->DR = c;
}
void USART1_IRQHandler(void)
{
/* Check if RXNE flag is set (Bit 5) */
if (USART1->SR & (1 << 5))
{
last_char = USART1->DR;
rx_count++;
USART1_SendChar(last_char);
}
}
int main(void)
{
USART1_Init();
NVIC_SetPriority(USART1_IRQn, 1);
NVIC_EnableIRQ(USART1_IRQn);
while (1)
{
idle_loops++;
}
}


Command
Running with Code Size Limit: 32K
Load "D:\\Embedded_System_lab-3\\Objects\\Task5.axf"
Include "D:\\Embedded_System_lab-3\\STM32F103.ini"
map 0x40000000,0x400FFFFF read write
_WDWORD(0x40021018,0x00000010)
_WDWORD(0x40011004,0x00000010)
ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess COVERAGE COVTOFILE
Watch 1
Name Value Type
last_char 0x41 'A' uchar
rx_count 0x00000001 uint
idle_loops 0x1106431C uint
(unsigned int)0x40013804 0x00000041 uint
(unsigned int)0x40013800 0x00000020 uint
(unsigned int)0xE000E204 0x00000000 uint
<Enter expression>
Call Stack + Locals Watch 1 Memory 1
Simulation t1: 237.71551883 sec L:33 C:1
CAP NUM SCRL OVR R/W

Conclusion : (Learning & Configuration)

In this lab, we successfully configured the USART1 peripheral of the STM32F103 microcontroller for serial communication. By directly manipulating the hardware registers, we learned how to enable peripheral clocks, configure GPIO pins (PA9 for TX and PA10 for RX), and set up the 9600 baud rate with an 8N1 frame format. Additionally, we implemented both polling and interrupt-driven methods for transmitting and receiving data. This practical session provided a solid understanding of how serial communication works at the register level and how to use the Watch window in Keil for debugging hardware operations.